

面向 LLM+OR 的 Operational Problem Complexity: 文献地图、定义框架与可验证研究路线

核心判断：你真正要定义的不是“solver 难度”，而是“一个 operational instance 对可靠求解系统造成的内生困难”

先给最重要的结论。

目前我没有找到 OR 或 LLM+OR 文献中已经形成共识的、**problem-intrinsic / model-independent** 的“**operational problem complexity**”定义。现有研究实际上分散在几条彼此相邻、但没有被统一起来的传统中：

一条是经典 OR 的 **model formulation / modeling process** 研究。Willemain 早在 1994–1995 年就把 formulation 明确视为 OR 实践中的核心“craft skill”，并通过专家 think-aloud 实验研究 modelers 如何在 problem context、model structure、realization、assessment、implementation 之间来回切换；但这类工作研究的是人如何建模，没有给问题实例定义一个可计算的 complexity measure。¹

第二条是数学优化与理论计算机中的 **representation/formulation complexity**。例如 extended-formulation/extension-complexity 文献研究一个组合优化多面体需要多大的 LP 表示，并证明某些经典组合优化多面体不存在小型 LP extended formulation；这是严格、model-independent 意义很强的“表示复杂度”，但它衡量的是**数学表示的最小规模**，并不直接衡量“从 business rules 理解出正确约束”对人或 LLM 有多难。²

第三条是 CSP、SAT 和算法选择领域的 **structural complexity / empirical hardness**。这里已经有非常成熟的思想：问题不能只由变量数、约束数决定，constraint interaction graph、treewidth 等结构会决定 tractability；另一边，SATzilla 等 empirical-hardness 工作则直接学习“instance features → solver runtime”，把 difficulty 定义为**可预测的 performance degradation**。这和你 Q5 的想法非常接近，但这些指标主要预测 algorithmic solving hardness，而不是 language-to-model formulation hardness。³

第四条，就是最近两年的 LLM+OR 研究。这里已经出现了你的研究问题的各个“碎片”：OPT-Engine 控制 problem scale、language complexity 和 constraint augmentation；ORAgentBench 提出六维 construction-time difficulty；NLCO 用 variable type、constraint family、global pattern、objective class 四层 taxonomy；LEAN-LLM-OPT 研究 input/data scale；MILP-LIB-NL 把 benchmark 推到工业级 MILP；R-ConstraintBench 特别观察 constraint interactions；IR2Solve 则开始把 semantic modeling 和 deterministic solver compilation 分离。**但是没有哪篇把这些碎片统一为一个经过因果验证、跨模型稳定、问题内生的 operational complexity。**⁴

因此我认为你的方向不仅成立，而且真正有论文空间。但我要先修正你现在思路中的一个关键点：

如果 Q1 坚持 problem-intrinsic / model-independent，那么 LLM accuracy 不应该直接成为 operational complexity 的定义；它应该成为 complexity 定义的主要验证信号。

假设直接定义

$$C(I) = -\log P(\text{GPT-5 solves } I),$$

那么换一个模型，complexity 就变了。这是 **model-relative empirical difficulty**，不是 problem-intrinsic complexity。

更合理的逻辑应该是：

instance structure $\rightarrow \mathbf{C}_{\text{op}}(I) \rightarrow \text{predict performance degradation across agents}$

也就是说，先从题目本身的语义、约束和求解结构定义一个 **complexity vector**；然后用多种 LLM、agent 和 solver 的 accuracy、feasibility、optimality gap、runtime 去验证它。

这是我认为这项工作的第一个理论支点。

现有文献究竟研究到了哪里

和你的问题最接近的文献地图

下面不是简单罗列 LLM+OR papers，而是按照它们“到底提供了哪一种 complexity 概念”重新分类。

工作	它实际上在测什么	是否 problem- intrinsic	是否直接研 究 modeling difficulty	对你最有价值的部分
Willemain, Operations Research, 1995	专家 formulation 的认知过程	部分	是	modeling 本身就是独立于求解的重要 intellectual task
Extension complexity 文献	一个优化问题/多面体的最小 LP 表示规模	强	间接	给 “model-independent mathematical representation complexity” 提供理论先例
CSP/treewidth 文献	constraint interaction structure	强	间接	给 constraint coupling 数学化提供最重要的理论模板
SATzilla / empirical hardness	instance features 对 solver runtime 的预测	否， algorithm- relative	否	给 “difficulty measure 必须预测 performance” 提供方法论模板
MAMO	solver-backed modeling correctness	部分	是	明确尝试把 modeling 与 solving 分离

工作	它实际上在测什么	是否 problem- intrinsic	是否直接研究 modeling difficulty	对你最有价值的部分
ORLM / IndustryOR	problem type、 industry、人工 difficulty	弱	是	建模问题的现实多样性；objective/ constraint augmentation
OptiBench	formulation correctness + solver- backed equivalence	部分	是	不应只看最终 objective value
OptMATH	controllable problem-data complexity	部分	是	天然适合 controlled complexity generation
Solver-Informed RL	solver feedback 对 formulation 学习	否	是	solver 可以做 verifier， 而非让 LLM 自己数值求解
LEAN-LLM-OPT	data/input scale、 problem class	部分	是	有 gold mathematical model，极适合你的 gold-model experiment
OPT-Engine	scale、PPL、 objective perturbation、 constraint augmentation	较强	非常直接	最接近 “controlled operational/modeling complexity”
NLCO	variable / constraint / global structure / objective taxonomy	较强	部分	比单纯 constraint count 更结构化
MIPLIB-NL	工业 MILP 规模和真实 formulation structure	较强	是	检验 “gold model 后 solver 一定简单” 最重要的反例来源
R- ConstraintBench	constraint quantity/ type/interaction	较强	直接针对 constraint reasoning	明确指出 interaction 比 单纯 graph depth 更关键
ORAgentBench	六维 modeling+solving burden	部分	非常直接	与你的 operational complexity 最接近，但 维度存在重叠
IR2Solve	semantic ModelIR → deterministic solver compilation	部分	非常直接	强力支持 “modeling 与 code generation 可 以解耦”

工作	它实际上在测什么	是否 problem- intrinsic	是否直接研 究 modeling difficulty	对你最有价值的部分
OR-Space	Build / Revise / Explain 全 lifecycle	否	是	给未来广义 operational complexity 留出 debugging/revision 维 度

Willemain 的研究已经说明 formulation 不是简单“把已知方程翻译成代码”：专家会不断在结构理解、实现与 assessment 间切换。⁵ MAMO 更直接地说，它使用 solver 的目的之一正是将 **modeling 与 solving 分离**，从而更集中地评估 mathematical modeling。⁶

ORLM 对 optimization modeling 的形式化也非常值得注意。它把一个自然语言 OR problem p 到 mathematical model m 和 solver program c 的过程写成

$$f: p \rightarrow (m, c),$$

并指出 m 包含 formal objectives and constraints，而 c 利用成熟 solver 完成求解。换句话说，这篇文献本身已经隐含了一个 **semantic formulation stage 与 executable realization stage 的分解**。⁷

2026 年的 IR2Solve 又把这个逻辑推得更远：它让 LLM 只负责一次 **semantic call，产生 schema-constrained ModelIR**，之后 verification 和 IR-to-Gurobi compilation 都是 deterministic 的。作者明确说这一分解把 natural-language interpretation 与 solver realization 分开，并有助于区分 semantic modeling errors 和 program-construction failures。⁸

这对你的 Q9 非常重要：

“对 LLM 来说，正确表达全部 feasibility conditions 比把已经正确 formalize 的模型交给 solver 求 optimum 更难。”

现在已经不能说这只是一个直觉了。它至少有三层已有证据支撑：

MAMO: isolate modeling from solving

OPT-Engine: constraint augmentation sharply hurts SIR

IR2Solve: semantic generation + deterministic compilation works

MAMO 明确把 solver 用来隔离 modeling；OPT-Engine 在不改变 asymptotic complexity、只加少量 constraints 的情况下观察到明显 accuracy decline；IR2Solve 则证明 solver-code realization 可以大量 deterministic 化。⁹

这已经是一条相当连贯的 research hypothesis。

但现有“complexity”定义有三个共同缺陷

首先，很多工作仍然把 **scale 当 complexity proxy**。LEAN-LLM-OPT 的 Large-Scale-OR 平均输入约 47,269 tokens，并观察到多种模型随 input size 增大而 modeling accuracy 下降；MILP-LIB-NL 更进一步，把实例推进

到 $10^3-10^6 + \text{variables/constraints}$ 。它们非常有价值，但 scale 本身不是你想要的 semantic/modeling complexity。 ¹⁰

第二，很多工作使用**人工 difficulty label 或高度相关的 rubric**。ORAgentBench 已经意识到 size 不足以代表 difficulty，提出 solving strategy requirement、formulation structure、constraint coupling、dynamic structure、data scale、problem understanding 六维；但作者自己在 regression analysis 中发现 constraint coupling 与 formulation/dynamic structure 有相关性，并明确说 adjusted impacts 是 descriptive attribution、不是 causal estimate。 ¹¹

第三，很多 benchmark 的 correctness 仍然高度依赖**最终 objective value**。LEAN-LLM-OPT 自己专门提供了“structurally incorrect model with a correct optimal solution”的 appendix，这实际上揭示了一个很关键的问题：**一次 instance 上算出正确 optimum，并不等价于 model semantics 正确**。 ¹² OptiBench 之所以强调 equivalence-detection evaluation，也是因为单纯 objective matching 不足以可靠衡量 formulation correctness。 ¹³

这三个缺口正好给你的工作留下空间。

OPT-Engine、ORAgentBench 和 LEAN-LLM-OPT 应该怎样解读

OPT-Engine 确实调用 solver，而且它的“solver guarantees optimality”必须加条件

你的小问题 a 的答案非常明确：

是。OPT-Engine 的 SIR (Solver-Integrated Reasoning) 明确调用 external solver。

作者将 SIR 定义为：LLM 先把自然语言问题翻译成 variables、objectives、constraints，然后 interfacing with external solvers such as Gurobi or COPT。实验里，SIR 的 terminal reasoning step 是 executable code snippet，随后在 external execution solver 中运行得到 objective value。 ¹⁴

因此 OPT-Engine 比较的并不是：

*LLM*直接算答案 vs *LLM*直接算答案

而是：

$$PTR : NL \rightarrow LLM \rightarrow solution$$

与

$$SIR : NL \rightarrow LLM \rightarrow formulation/code \rightarrow solver \rightarrow solution.$$

更有意思的是，作者在 error decomposition 中明确写：

SIR 的数值 execution 被 external solver 接管；如果模型捕获了正确 logical grounding，即 constraints and objectives，则 solver guarantees optimality。 ¹⁵

你 Q7 的怀疑是对的，但需要非常精确地表达。

这个 statement 在 OPT-Engine 的 experimental regime 内可以成立；把它升级成普遍 OR 命题就不成立。

它隐含至少四个条件：

gold/correct formulation
+solver supports the formulation class
+solver successfully terminates to required optimality tolerance
+available computation is sufficient

才可以推出：

solver-returned solution is optimal for that model.

而 OPT-Engine 自己在 limitations 中实际上已经收回了这个命题的普适性：作者明确承认 exact modeling/solving 对 large-scale NP-hard problems 可能 computationally prohibitive，因此实践中会偏好 heuristics；并建议未来报告 optimality gap、runtime、time budgets、heuristic selection。同时当前 benchmark 只覆盖 linear LP/MIP structure，没有 nonlinear、stochastic、dynamic programs。 ¹⁶

所以我会把 OPT-Engine 的命题重写成：

Correct-model sufficiency hypothesis

对于 **solver-supported、exactly solvable、benchmark-scale** 的 optimization instances，一旦 semantic formulation 正确，剩余 solver stage 的 error probability 通常远低于 NL-to-formulation stage。

这个命题是很有研究价值的。

但下面这个更强的命题：

correct formulation $\Rightarrow$ easy/guaranteed operational solution for arbitrary OR instance

是不能直接成立的。

routing、scheduling、industrial MILP、stochastic optimization、large-scale combinatorial optimization、nonconvex NLP 恰好就是最需要用来检验这个强命题的反例族。OPT-Engine 自己已经指出 large-scale NP-hard、nonlinear、stochastic、dynamic 是其当前 scope 外的重要延伸。 ¹⁶

这也意味着你的 gold-model experiment 绝不能只在 LP/MILP toy benchmark 上做，否则很容易“证明一个 benchmark artifact”。

OPT-Engine 对 constraint complexity 的证据比它自己的 complexity definition 更重要

我认为 OPT-Engine 最有价值的其实不是它原有的 scaling，而是 Section 5 的 controlled intervention。

它做了三类变化：

linguistic complexity, objective perturbation, constraint augmentation.

对 language complexity，它通过增加 lexical/syntactic difficulty、以 perplexity 衡量，而保持 underlying numeric instance 和 constraint set 相同，accuracy 基本稳定。 ¹⁷

对 objective，它只是加一个与 decision variables 无关的 constant：

$$\min_x f(x) \rightarrow \min_x f(x) + K,$$

accuracy 也基本不受影响。 17

但对 constraints，它在**不引入新变量、只添加 $O(1)$ 个额外约束、保持 formal problem class 与 asymptotic complexity 不变**的情况下，看到明显 performance degradation。作者因此明确解释为 modeling burden 从 computation 转移到了 semantics。 18

这其实已经非常接近你要做的第一代实验：

same computational class + similar scale + different semantic constraint structure

然后观察：

$$\Delta P(\text{correct formulation}).$$

但是 OPT-Engine 现在还没有回答你的真正问题：

为什么某组 extra constraints 比另一组更难？

“constraint 数变多” 仍然不是答案。

这就是 constraint coupling metric 的空间。

ORAgentBench 的 end-to-end 绝对不等于 “把题发给 LLM，让它直接报答案”

你的小问题 b：

不是。

ORAgentBench 定义一个 task 为

$$\tau = (\mathcal{D}, \mathcal{C}, q, \mathcal{E}, \mathcal{V}),$$

其中 $\mathcal{D}$ 是 data， $\mathcal{C}$ 是 configuration， q 是 natural-language operational brief， $\mathcal{E}$ 是 execution environment， $\mathcal{V}$ 是 hidden validator。Agent 收到

$$I_\tau = (\mathcal{D}, \mathcal{C}, q)$$

之后，要生成 executable program：

$$p_\theta = A_\theta(I_\tau),$$

再在 sandbox 中执行：

$$s = \text{Exec}_\mathcal{E}(p_\theta; \mathcal{D}, \mathcal{C}),$$

最后 validator 对 decision artifact s 检查 schema、hard-constraint feasibility 和 objective quality。 19

ORAgentBench 甚至明确把 agent 内部行为分成：

$$\begin{aligned}\phi_{\theta,\tau} &= \text{modeling strategy}, \\ \psi_{\theta,\tau} &= \text{solving strategy}.\end{aligned}$$

它强调 realistic OR task 要同时设计 representation 和 computational procedure。 20

所以它的 “end-to-end” 应该理解成：

operational artifacts $\rightarrow$ understanding $\rightarrow$ model/strategy $\rightarrow$ code $\rightarrow$ solver/search $\rightarrow$ decision artifact

而不是 PTR 式的：

$$NL \rightarrow LLM \rightarrow \text{answer}.$$

这一点非常重要，因为你的 operational complexity 本质上是在试图研究前一条 pipeline 中 **difficulty 到底产生在哪里**。

ORAgentBench 六维 rubric 的确存在定义重叠，而且论文自己已经暴露了这个问题

你的小问题 c 的判断也是基本正确的。

论文列出的六维是：

{solving strategy requirement, formulation structure, constraint coupling, dynamic structure, data scale, problem complexity}

作者说明这些维度用于 prospective construction-time difficulty control，并强调 instance size alone 不足以代表 difficulty。 21

但在公开论文正文和 appendix 的文本中，我没有看到一张能够使第三方独立复现的、类似下面这种 granular rubric：

$$\begin{aligned}\text{constraint coupling} &= 0 : \dots \\ &= 1 : \dots, \quad = 2 : \dots, \quad = 3 : \dots.\end{aligned}$$

论文后面的统计分析却明确把每个 x_{id} 当成 0-3 的 construction score，并拟合

$$D_i = \alpha + \sum_{d=1}^6 \beta_d^{\text{multi}} x_{id} + \epsilon_i,$$

再用 marginal slope shrinkage 得到 adjusted impact。作者同时明确强调：这是 **descriptive attribution, not a causal estimate**。 22

更关键的是：

作者自己发现 constraint coupling 与 formulation structure、dynamic structure 有 correlation，因此其 joint coefficient 被压低。 23

所以你感觉 “六个维度定义可能重叠” 不是错觉。

例如：

- dynamic structure 往往天然产生跨时间 constraints；
- 跨时间 constraints 又会增加 constraint coupling；
- 高 coupling 常常需要更复杂 formulation；
- complex formulation 又往往需要 specialized solving strategy。

于是：

dynamic $\rightarrow$ coupling $\rightarrow$ formulation $\rightarrow$ solving strategy

四个 rubric 很容易不是 orthogonal causes，而是同一 causal chain 上不同位置的变量。

这正是你的工作可以明显超过 ORAgentBench 的地方：

不要从“人工打六个 difficulty 分数”开始，而应从一个明确的 structural representation 出发，把各维度定义成可计算量，再用 controlled intervention 验证。

LEAN-LLM-OPT 给你的 gold-model experiment 提供了非常好的现成材料

Large-Scale-OR 的每个 instance 本身就包括：

natural-language query + datasets

以及 label：

ground-truth mathematical model + optimal value.

这是作者明确的数据设计。 ¹²

而且这个 benchmark 比早期 toy benchmark 更接近你 Q3 的完整 operational instance：problem description 与 datasets 分离；101 个测试实例覆盖 medical、e-commerce、supply-chain 等应用，并包含 network revenue management、resource allocation、facility-location 等常见类别和 mixture problems。

¹²

因此它几乎天然支持：

Condition A: $NL + D \rightarrow model + code \rightarrow solver$

与

Condition B: $gold\ model + D \rightarrow code \rightarrow solver$

的 intervention。

更有意思的是 Large-Scale-OR 的平均 input length 达到约 47,269 tokens，而 NL4OPT、MAMO Complex、IndustryOR 分别远小得多；论文观察到很多模型随输入长度增加 modeling accuracy 降低。 ¹²

所以它还允许你把：

$C_{data/context}$

和

$$C_{\text{semantic/modeling}}$$

初步拆开，而不是把两者混为一个“difficulty”。

LLM+OR benchmark 中实际出现的问题类别，以及 gold model 后到底还难不难

你 Q10 的问题可以明确回答：

是的，你列的 LP/MILP → scheduling → routing → network optimization → facility location → inventory/revenue management 基本都已经出现在当前 LLM+OR benchmark 生态中。

但要强调的是：

不是每一个 benchmark 都覆盖这些类别，而是“整个 benchmark ecosystem”加起来覆盖了这些类别。

例如 OPT-Engine 只有十个 canonical LP/MIP classes：LP 侧包括 inventory、portfolio allocation、production、transportation、pollution control；MIP 侧包括 TSP、knapsack、bin packing、job-shop scheduling、minimum-cost network flow。它明确不包含 stochastic、dynamic、nonlinear。 ²⁴

Large-Scale-OR 则明显扩展到 network revenue management、resource allocation、transportation、facility location、assignment 和 mixed problems。 ¹²

IndustryOR 覆盖 LP、NLP、IP、MIP 和“other”五类 optimization models。 ²⁵

NLCO 更宽，覆盖 43 类 combinatorial optimization problem，并专门按 variable types、constraint families、global patterns、objective classes 组织，包括 routing、scheduling、packing、assignment、facility-location、graph/network 等大量结构。 ²⁶

ORAgentBench 又进一步允许 direct LP/MIP、optimization+repair、decomposition、heuristics、network flow、matching、shortest path、constraint programming 等不同 solving strategies。 ²²

因此，你的第一版 benchmark **不需要凭空创造新的 OR problem universe**；完全可以先覆盖当前 LLM+OR papers 已经使用的 classes。

下面这个排序不是“全球 OR 行业使用频次排名”，而是我建议你按照 **LLM+OR benchmark 覆盖度 + 对你 hypothesis 的诊断价值** 来排。

问题族	一个最小样例	formulation 最容易错在哪里	solver code 标准化程度	gold model 给出后, 还明显需要 solving strategy 吗?	对你的 hypothesis 的意义
LP / resource allocation / production	两种产品共享人工与原料, 最大化利润	单位、capacity direction、需求上下界、objective coefficient	很高	通常很少	最强 positive control; 应最支持 modeling-dominates
Transportation / basic network flow	三个仓库向四个客户配送	flow conservation、supply/demand balance、direction	很高	单 commodity 通常很少	若 gold model 后仍失败, 说明 pipeline/code 有明显问题
Assignment / matching	员工分配任务, 一人一岗	one-to-one / at-most-one / compatibility	很高	通常很少	适合研究 constraint omission 和 indexing
Facility location	决定开哪些仓库并给客户分配仓库	open-assignment linking、capacity、fixed+variable cost	高	中小实例较少; 大规模可能明显	开始出现 formulation 与 MIP hardness 分离
Knapsack / packing / bin packing	将物品放入容量有限的箱子	assignment、capacity、activation、symmetry	高	小规模少; bin packing 大规模可能有 solver burden	测试 “canonical template familiarity”
Scheduling	多 job、多 machine、有 precedence 与 no-overlap	时间索引、precedence、calendar、disjunctive/no-overlap	中等	经常需要	第一个强反例族: 正确语义不一定意味着标准化 solver path
Routing / TSP / VRP	有容量与 time window 的车辆配送	subtour、flow continuity、capacity、time-window coupling	中等	大实例经常需要	最重要反例之一: modeling 与 computational strategy 都可能重要
Inventory / multiperiod planning	多周期补货、库存平衡、backlog	跨期 state balance、boundary conditions、lead time	高到中	deterministic 版本通常较少; 复杂版本增加	很适合研究 temporal coupling

问题族	一个最小样例	formulation 最容易错在哪里	solver code 标准化程度	gold model 给出后，还明显需要 solving strategy 吗？	对你的 hypothesis 的意义
Revenue management / NRM	有容量约束的航班网络控制产品销售	network incidence、choice behavior、capacity coupling	中等	取决于模型；大规模/choice-based 可能明显	数据理解、model structure 与 solving 可以同时出现
Stochastic / robust optimization	demand scenarios 下先决策、后 recourse	scenario indexing、nonanticipativity、chance/robust constraints	中低	往往明显	对“gold model 后就容易”构成强 stress test
Nonlinear / nonconvex optimization	非线性生产或风险-收益模型	convexity、nonlinear relation、domains	中低	明显可能需要 solver choice / initialization / global-vs-local	直接否定 universal solver-guarantee interpretation
Dynamic / replanning operational problems	disruption 后保留已冻结决策、重新调度	state persistence、freeze window、cross-stage validity	低到中	明显需要	更接近 broad operational problem solving complexity

这些差异解释了为什么你基于 LP 得出的直觉 **方向是对的，但不能直接外推到整个 OR。**

对于 canonical LP，流程确实接近：

correct model → standard solver API → correct optimum

只要 formulation 与 data 正确，后面的 solver invocation 几乎是机械性的。这也是 MAMO、ORLM、IR2Solve 这类工作能够把 modeling 和 solving 强烈解耦的原因。²⁷

但 scheduling 与 routing 不一样。

OPT-Engine 自己观察到 TSP 在 direct reasoning 下，随着规模增加，模型会从 enumeration 转向 nearest-neighbor、cheapest-insertion 等 heuristic behavior；作者由此强调大规模组合优化中 exact solving 与 heuristic solving 的差别。¹⁵ ORAgentBench 更进一步，把 decomposition、warm starts、restricted re-optimization、repair、rolling horizon、heuristic search、specialized network/CP methods 都视作现实 solving strategy 的组成部分。²²

因此你的 gold-model experiment 最有价值的地方不是证明一个绝对结论：

“solver complexity 不重要。”

而是找出一个 **phase boundary**:

在哪些 problem classes / scales 下 $C_{\text{solve}} \approx 0$, 在哪些情况下 $C_{\text{solve}} > 0$?

这比“建模重要还是求解重要”这个二元问题学术价值更高。

可以想象最后你的 empirical finding 是:

LP/resource allocation/network flow : $C_{\text{model}} \gg C_{\text{solve}}$

facility location/small MILP : $C_{\text{model}} > C_{\text{solve}}$

scheduling/routing : $C_{\text{model}} \approx C_{\text{solve}}$ at sufficient scale

stochastic/nonconvex/industrial MILP : C_{solve} cannot be ignored.

这是一个比 OPT-Engine 当前 “solver guarantees optimality” 更一般、也更 OR-correct 的理论图景。OPT-Engine 自己的 limitation 已经暗示了这个分界。 ¹⁶

MIPLIB-NL 对这里尤其重要。它从真实 MIPLIB 2017 MILPs 重构了 223 个 NL-to-optimization instances, 并让自然语言 specification 与原始 mathematical formulation 一一对应, 规模达到 10^3 – 10^6 + variables/constraints; 这正适合检验 “即使 gold formulation 已知, industrial-scale solving 是否仍然形成 residual difficulty” 。 ²⁸

我建议怎样正式定义 Operational Complexity

先明确对象, 而不是直接定义一个分数

你的 Q3 选择的是完整 operational instance:

$$I = (q, D, R),$$

其中:

- q : natural-language operational objective / brief;
- D : instance data;
- $R = \{r_1, \dots, r_m\}$: business rules / operational feasibility requirements.

但因为你又要求 Q1:

problem-intrinsic / model-independent,

所以复杂度不能依赖某一个 Gurobi formulation.

定义:

$$\mathcal{M}(I) = \{M : M \text{ is a semantics-preserving formalization of } I\}.$$

不同的 M 可以使用不同变量、auxiliary variables、linearization、time-index formulation、network formulation 等, 但它们表达同一个 underlying operational problem.

理想的复杂度应该满足：

$$C(I) = C(I')$$

只要 I 和 I' 是 semantic-equivalent paraphrases；并且尽量满足

$$C(M_1) \approx C(M_2) \quad \forall M_1, M_2 \in \mathcal{M}(I),$$

而不是因为某个建模者用了 5000 个 auxiliary variables，问题就突然“更复杂”。

这就是 **model-independent** 最重要的含义。

不建议第一版把所有东西塞成一个 scalar

我建议第一篇论文先定义：

$$\mathbf{C}_{\text{op}}(I) = (L_I, \kappa_I, \delta_I, H_I, G_I, \rho_I)$$

其中前四个构成 **optimization modeling complexity core**：

$$\mathbf{C}_{\text{model}} = (L, \kappa, \delta, H)$$

而

$$G = \text{data/context grounding burden}$$

和

$$\rho = \text{residual solving hardness}$$

先作为独立维度。

这与你 Q4 “先 vector，之后再研究 aggregation” 高度一致。

同时它让你的两个 narrative 都成立：

对于 OR：

$$\text{operational complexity} = \text{modeling structure} + \text{residual solving burden}.$$

对于 ML/LLM benchmark：

$$\text{instance features} \rightarrow \text{predict accuracy degradation}.$$

Semantic obligation load：不要简单数 constraints

首先建立一个非常关键的中间概念：

$$\mathcal{O}(I) = \{o_1, \dots, o_m\},$$

叫做 **semantic obligations**。

一个 obligation 不是 solver 里的 “一行 constraint”，而是：

一个独立的 operational requirement；如果不满足它，solution 在业务语义上就是错误的。

比如：

“每个客户必须恰好被一辆车服务”

是一个 semantic obligation，即使最后写成：

$$\sum_k \sum_j x_{ijk} = 1 \quad \forall i.$$

这不是因为有 100 个客户就算 “100 个不同语义 constraints”。

再比如：

“如果员工上夜班，第二天不能上早班”

也是一个 semantic obligation family。

这样就避免：

$$\text{constraint count} \equiv \text{complexity}$$

这种明显不可靠的定义。

定义 semantic obligation load：

$$L(I) = \log(1 + |\mathcal{O}(I)|)$$

可以作为第一版简单 measure。

但我认为 L 不会是最有解释力的维度。

真正关键的很可能是下面的 κ 。

Constraint coupling：不要研究 “文本 embedding 上的语义耦合”，要研究 operational-rule coupling

这是我对 Q8 最强的建议。

不要把主要研究对象叫 natural-language semantic coupling。

因为如果你定义：

$$\text{coupling}(r_i, r_j) = \cos(\text{embedding}(r_i), \text{embedding}(r_j)),$$

它很可能既不稳定，也没有 OR 意义。

两个语句：

“Truck capacity cannot exceed 10 tons.”

和

“Each customer must be visited exactly once.”

文本语义相似度可能很低，但它们在 VRP 的 solution structure 中高度相互作用。

反过来，两个文本表达很相似的 constraints 可能约束的是完全独立的两个 geographic region。

所以你真正需要的是：

semantic-to-structural constraint coupling

或者我更喜欢叫：

Operational Constraint Coupling

建立一个 **Semantic Obligation Hypergraph**：

$$\mathcal{H}_I = (V, E).$$

其中 V 不是 solver scalar variables，而是 canonical semantic decision entities，例如：

{customer assignment, vehicle usage, time, capacity, inventory state, facility opening}.

每个 semantic obligation o_r 形成一个 hyperedge：

$$e_r \subseteq V.$$

举例，VRP：

o_1 : each customer served once

可能连接：

{customer, vehicle, route}.

而

o_2 : vehicle capacity

连接：

{customer, vehicle, route, capacity}.

二者共享大量 decision semantics，因此高度 coupled。

然后你可以构造 obligation-overlap graph：

$$W_{rs} = \frac{|e_r \cap e_s|}{|e_r \cup e_s|}, \quad r / s. \quad =$$

第一版 coupling vector 可以使用：

$$\kappa(I) = (\bar{w}, \lambda, \tau)$$

其中：

$$\bar{w} = \frac{2}{m(m-1)} \sum_{r < s} W_{rs}$$

是平均 overlap；

$$\lambda = \frac{\lambda_{\max}(W)}{m-1}$$

是 normalized spectral coupling；

$$\tau = \frac{\text{tw}(G_I)}{|V(G_I)| - 1}$$

是 [normalized treewidth-like global interaction measure](#)。

这里 treewidth 的思想并不是凭空创造。经典 CSP literature 很早就表明 constraint network 的 graph structure、特别是 bounded-treewidth 类型结构，与 tractability 密切相关；这正说明“约束数量相同但 interaction topology 不同”可以导致完全不同的困难。 ²⁹

不过我要强调：

你不应该一开始宣布 treewidth 就是 operational complexity。

你的研究问题更有趣：

Which coupling statistic best predicts LLM formulation failure?

候选可能是：

[mean overlap](#), [spectral radius](#), [treewidth](#), [hubness](#), [cycle density](#).

然后用实验选择。

这就从“拍脑袋设计 complexity rubric”升级成：

寻找具有 predictive validity 的 structural complexity measure。

这非常符合你 Q5。

而且 R-ConstraintBench 已经给出一个非常好的 preliminary signal：在 RCPSP 上，它逐渐加入 precedence、downtime、temporal windows、disjunctive constraints；实验观察到仅 precedence DAG 时强模型接近 ceiling，但当多类 constraints interaction 后 feasibility 明显 collapse，作者指出 interaction 而不是单纯 graph depth 是主要 bottleneck。 ³⁰

这与你的 hypothesis 几乎正面对齐。

Compositional depth: 区分“约束多”与“必须组合推理很多层”

定义:

$$\delta(I) = \text{maximum semantic dependency depth.}$$

例如:

如果 facility A 开启, 则至少配置两名员工;
如果配置夜班员工, 则必须满足休息规则;
如果当天 demand 超过 threshold, 则必须开启 overflow facility;
overflow facility 只能在主 facility 达到 capacity 后启动。

这不是单纯四条 independent constraints, 而是 dependency chain。

构造 directed dependency graph:

$$o_i \rightarrow o_j$$

表示正确 instantiate o_j 依赖理解 o_i 的 outcome/state。

如果 DAG:

$$\delta = \max_{\pi} |\pi|.$$

如果存在 cycles, 可以先做 strongly connected component condensation, 再计算 DAG depth, 并把 cycle/feedback 作为额外 statistic。

这会比“language perplexity”更接近你想研究的 semantic/modeling burden。OPT-Engine 已经观察到单纯提高 PPL 并没有显著降低 accuracy, 因此 surface linguistic complexity 至少不是一个足够好的 complexity proxy。 ¹⁷

Constraint heterogeneity: 相同数量、相同 coupling, 也可能因为 rule types 不同而难度完全不同

定义 constraint semantic type:

$$t(o_i) \in \{\text{capacity, balance, assignment, precedence, logical, temporal, activation, nonanticipativity, chance, } \dots\}.$$

然后:

$$p_k = \frac{|\{o : t(o) = k\}|}{|\mathcal{O}|}$$

定义:

$$H(I) = - \sum_k p_k \log p_k.$$

这不是说 entropy 一定就是最佳指标，而是它提供一个清晰的可检验 hypothesis：

在 constraint count 和 coupling 相同的情况下，heterogeneous rule families 是否比 homogeneous rules 更容易造成 formulation error？

NLCO 已经沿着类似方向做了四层 taxonomy：variable types、constraint families、global patterns、objective classes，并观察到不同 structural categories 的难度具有系统差异，例如 graph-structured 与 bottleneck-objective tasks 更容易失败。³¹

因此这不是毫无先例的指标。

Data grounding complexity 应该保留，但暂时不要让它污染 modeling core

你认为 data understanding 相对 optimization modeling complexity 次要，我认为可以把它作为 empirical hypothesis，而不是预设结论。

定义：

$$G(I) = \text{data-to-rule grounding burden}.$$

它可以包含：

$$G = (n_{\text{files}}, n_{\text{schemas}}, n_{\text{cross-file joins}}, n_{\text{relevant fields}}, \text{context length}).$$

LEAN-LLM-OPT 已经清楚显示 large-scale optimization modeling 的 input data 与 problem descriptions 可以非常长，并且随着 input scale 增长 modeling accuracy 明显下降。¹² ORAgentBench 也刻意使用 multi-file operational artifacts、cross-file dependencies，而不是完全 normalized problem instance。³² OR-Space 更进一步把 business documents、structured data、code artifacts、solver outputs 放到 persistent workspace 中。³³

因此 G 不能完全删掉。

但可以设计实验问：

$$\Delta R^2 = R^2(C_{\text{model}} + G) - R^2(C_{\text{model}}).$$

如果这个值很小，你就有证据说：

modeling structure explains most operational failure beyond raw data/context burden.

而不是事先凭直觉删掉 data understanding。

Residual solve hardness 不应该简单等于 solver runtime

这是 Q1/Q7/Q12 中最容易混淆的一点。

如果你要求 problem-intrinsic, 那么:

Gurobi runtime

不是严格 problem-intrinsic, 因为换 solver、换 formulation、换 hardware, runtime 都变。

SATzilla/empirical-hardness literature 恰好证明了这一点: 它学习的是 instance features 对**特定 algorithms 的 runtime** 的预测, 并据此选择 solver。它是一种非常成功的 empirical difficulty, 但本质上是 algorithm-relative。 ³⁴

所以第一篇论文不要把 raw solver time 直接混进 operational complexity scalar。

我建议定义一个**validation-side residual hardness**:

$$\rho_B(I) = \inf_{M \in \mathcal{M}(I), s \in \mathcal{S}} h_B(M, s, I),$$

其中:

- $\mathcal{M}(I)$: 允许的 faithful formulations;
- $\mathcal{S}$: 预先规定的 standard solver family;
- B : 固定 resource budget;
- h_B : 例如 optimality gap、nodes、time-to-proof 的 normalized combination。

严格来说它仍然是 protocol-relative, 所以我建议论文语言里称:

empirical residual solve hardness

而不要声称是 absolute computational complexity。

这样你可以同时保留:

C_{model}

作为主要 problem-intrinsic contribution,

又保留:

ρ

去检验 “computational side 到底什么时候不可忽略”。

最关键的验证实验: Gold-Model Intervention

我认为这是整个项目最应该先做的实验, 因为它不仅验证你的 hypothesis, 还决定后面的 complexity 到底需不需要包含 computational component。

不要只做两个 condition, 应该做四层 intervention

你现在考虑:

$$NL \rightarrow model + code$$

versus

$$gold\ model \rightarrow code.$$

我建议扩展为四层。

Full operational input

$$A : (q, D, R) \xrightarrow{\text{LLM/Agent}} \hat{M}, \hat{c} \xrightarrow{\text{solver}} \hat{y}.$$

这是正常 end-to-end benchmark。

Gold mathematical model

$$B : (M^*, D) \xrightarrow{\text{LLM}} \hat{c} \xrightarrow{\text{solver}} \hat{y}.$$

这里消除了 semantic formulation burden，但保留 LLM code generation。

Gold structured model + deterministic compiler

$$C : M^* \xrightarrow{\text{deterministic compiler}} c^* \xrightarrow{\text{solver}} y.$$

这消除 code-generation noise。

IR2Solve 已经证明这种设计是现实可行的：LLM 生成 ModelIR 后，verification 与 IR-to-Gurobi compilation 可以完全 deterministic。 ⁸

Gold executable model

$$D : c^* \xrightarrow{\text{solver}} y.$$

这是纯 solving condition。

这样可以近似分解：

$$P_A = P(\text{E2E success}),$$

$$P_B = P(\text{code+solve success} \mid M^*),$$

$$P_C = P(\text{solve success} \mid \text{structured gold model}),$$

$$P_D = P(\text{solver success} \mid \text{gold executable model}).$$

于是 modeling contribution：

$$\Delta_{\text{model}} = P_B - P_A$$

LLM code-generation contribution：

$$\Delta_{\text{code}} = P_C - P_B$$

residual solve failure:

$$\Delta_{\text{solve}} = 1 - P_D$$

或者使用 odds scale:

$$\Delta_{\text{model}}^{\text{logit}} = \text{logit}(P_B) - \text{logit}(P_A).$$

如果你发现:

$$P_A = 0.45, \quad P_B = 0.96, \quad P_C = 0.99, \quad P_D = 1.00,$$

那么你几乎可以非常有力地说:

$$\text{semantic formulation dominates operational failure}$$

至少在该 problem regime 中。

但如果 routing / scheduling / MIPLIB-NL 上是:

$$P_A = 0.30, \quad P_B = 0.70, \quad P_C = 0.74, \quad P_D = 0.78,$$

那么就说明:

$$C_{\text{solve}}$$

不可忽略。

这就是你真正要寻找的 boundary。

最适合做这个实验的 benchmarks

第一层我会优先:

Large-Scale-OR。因为它明确提供 NL+datasets 与 ground-truth mathematical model+optimal value。

12

OptMATH。它的数据生成方向本来就是:

$$MF \rightarrow PD \rightarrow NL,$$

即从 curated mathematical formulations 出发, 生成 controllable-complexity problem data, 再 back-translate 成 natural language, 因此 gold MF 天然存在。 35

MIPLIB-NL。它提供自然语言 specification 与真实 MIPLIB mathematical formulation/solver representation 的一一对应, 特别适合作为 industrial-scale stress test。 36

第二层可以纳入:

MAMO, 因为它的 benchmark philosophy 本身就是使用 solver isolate modeling from solving。 37

IndustryOR，因为它有真实 OR cases 与多类 LP/NLP/IP/MIP，适合测试跨 problem type。 38

OPT-Engine，因为它提供 controlled scaling，可以把 gold-model experiment 和 constraint augmentation 结合。 17

ORAgentBench 反而不应该成为第一版 gold-model experiment 的主 benchmark，因为它的目标就是 artifact-to-decision full workflow，并刻意让 modeling strategy 与 solving strategy 都是 latent decisions；它更适合后续验证 broad operational complexity，而不是第一阶段 isolation。 20

不能只看 final objective value

这一点非常重要。

假设真实模型：

$$\max f(x) \quad s.t. \quad x \in \mathcal{F}.$$

LLM 生成：

$$\max f(x) \quad s.t. \quad x \in \hat{\mathcal{F}}.$$

完全可能恰好存在：

$$x^* \in \mathcal{F} \cap \hat{\mathcal{F}}$$

而且：

$$f(x^*) = f(\hat{x}^*),$$

于是一次 instance 上 objective match，但模型其实错了。

LEAN-LLM-OPT 已经明确给出 structurally incorrect model nevertheless producing the correct optimal solution 的案例；OptiBench 也正因为这一问题强调 equivalence detection。 39

因此 modeling correctness 至少要有三层：

objective fidelity,
constraint/obligation fidelity,
solution-set behavior.

我尤其建议加入一个新的 **counterfactual activation test**。

假设一个错误模型漏掉某条 constraint，但是当前 data 上这条 constraint 是 slack，所以 optimum 不变。

那就改变 parameter：

$$D \rightarrow D^{(1)}, D^{(2)}, \dots, D^{(K)}$$

故意让这条 constraint 成为 active/binding。

如果两个模型真的 semantically equivalent, 应在 counterfactual instances 上继续保持一致; 如果只是偶然得到同一个 optimum, 就会暴露。

定义:

$$E_{\text{cf}}(M, \hat{M}) = \frac{1}{K} \sum_{k=1}^K \mathbf{1}[\text{Sol}(M, D^{(k)}) \simeq \text{Sol}(\hat{M}, D^{(k)})].$$

我认为这比 exact-string/model matching 强得多, 也比 single-instance optimal-value matching 可靠得多。

它甚至可能单独成为一个非常好的 benchmark contribution。

Feasibility 应该怎样重新定义

你提到 “feasibility 相较 optimality 更难满足” 和 weak feasibility, 这里需要区分 direct-solving 与 solver-integrated 两种情况。

ConstraintBench 是 direct optimization: LLM 不调用 solver, 直接输出 solution。它发现最佳模型 constraint satisfaction 只有约 65%, 而 feasible solutions 的 objective 通常已经达到 Gurobi optimum 的 89–96%; 没有模型在 “feasible 且距最优 0.1% 内” 的联合指标超过 30.5%。 ⁴⁰

但在 solver-integrated setting 中, 只要 generated model 被 solver exact solve:

$$\hat{x} \in \hat{\mathcal{F}}$$

几乎是 solver 的基本职责。

真正的问题是:

$$\boxed{\hat{\mathcal{F}}} / \square = \mathcal{F}$$

也就是:

solver-feasible $\neq$ operationally feasible。

这可以分成三种 semantic modeling error:

Under-constrained

$$\mathcal{F} \subset \hat{\mathcal{F}}.$$

LLM 漏掉 business rule。

solver 得到:

$$\hat{x} \in \hat{\mathcal{F}},$$

但可能:

$$\hat{x} \notin \mathcal{F}.$$

这就是最典型的“solver 说 feasible，但业务上 invalid”。

Over-constrained

$$\hat{\mathcal{F}} \subset \mathcal{F}.$$

LLM 多加或误解 constraints。

solver solution 可能 operationally feasible，但是：

$$f(\hat{x}) < f(x^*),$$

或者错误模型本身 infeasible。

Distorted feasible set

$$\hat{\mathcal{F}} \not\subseteq \mathcal{F}, \quad \mathcal{F} \not\subseteq \hat{\mathcal{F}}.$$

既漏约束又错约束。

我建议把这个概念叫：

Semantic Feasibility Fidelity

而不是 vague 的 weak feasibility。

例如定义 constraint-recall style measure：

$$SFF(M) = \frac{\sum_{o \in \mathcal{O}(I)} w_o \mathbf{1}[M \models o]}{\sum_{o \in \mathcal{O}(I)} w_o}.$$

或者 solution-based：

$$SFF_{\mu}(M) = \Pr_{x \sim \mu(\hat{\mathcal{F}})} [x \in \mathcal{F}].$$

这会把你所说的 constraint issue 从 anecdotal observation 变成数学对象。

Constraint coupling 最关键的实验不是 regression，而是 intervention

ORAgentBench 最大的 methodological weakness 恰好可以成为你的 methodological contribution。

他们有六维 scores，但这些维度 correlated，因此 post-hoc coefficient 很难解释为 causal influence。作者自己明确承认 regression 是 descriptive、not causal，并指出 coupling 与 formulation/dynamic structure correlation。 ⁴¹

你的实验应该反过来做：

factorial controlled benchmark generation

构造相同：

$$n_{\text{rules}}, \quad n_{\text{entities}}, \quad \text{constraint families}, \quad \text{token length}, \quad \text{solver runtime}$$

但改变 coupling topology。

例如四类：

Block-independent

$$o_1 - o_2 \quad o_3 - o_4 \quad o_5 - o_6$$

几组 rule 互不影响。

Chain

$$o_1 - o_2 - o_3 - o_4 - o_5 - o_6.$$

Hub

$$\begin{array}{c} o_2 \\ | \\ o_3 - o_1 - o_4 \\ | \\ o_5 \end{array}$$

一个 central rule 与大量 rules 共享 entities。

Dense

$$o_i \leftrightarrow o_j \quad \text{for many } i, j.$$

然后保持：

$$|\mathcal{O}| = \text{constant}$$

而让：

$$\kappa_{\text{block}} < \kappa_{\text{chain}} < \kappa_{\text{hub/dense}}.$$

如果观察到：

$$P(\text{correct modeling}) \downarrow \quad \text{as} \quad \kappa \uparrow,$$

并且：

$$T_{\text{solver}} \approx \text{matched},$$

那么你就获得了比 OPT-Engine “多几个 constraints accuracy 会下降” 强很多的证据：

constraint interaction structure causally increases LLM modeling difficulty

R-ConstraintBench 目前已经观察到 interaction 是重要 failure driver，但它仍主要通过不同 constraint families 顺序叠加来得到这个结论；你可以进一步做同数量、同 constraint types、只 rewiring coupling graph 的严格 causal intervention。 ³⁰

这很可能是整个项目最原创的 experimental design。

如何验证 complexity vector 真的是 “model-independent difficulty measure”

你的目标不能只是：

R^2 很高.

至少需要五种 validity。

Predictive validity

训练：

$$\text{logit } P(Y_{a,i} = 1) = \alpha_a + \boldsymbol{\beta}^\top \mathbf{C}_i + u_{\text{class}(i)}.$$

其中 a 是 agent/model, i 是 instance。

看 complexity vector 是否预测：

model correctness, operational feasibility, pass rate.

更强版本允许不同模型有不同 sensitivity：

$$\text{logit } P(Y_{a,i} = 1) = \alpha_a + (\boldsymbol{\beta} + \mathbf{b}_a)^\top \mathbf{C}_i + u_{\text{class}(i)}.$$

如果所有模型都对某个 dimension：

$$\beta_\kappa > 0$$

表现一致，那么 coupling 才有资格被称为 intrinsic difficulty axis。

Cross-model validity

用 models：

$$A_1, \dots, A_k$$

拟合 complexity-performance relationship,

然后在完全没参与 fitting 的新模型：

$$A_{k+1}$$

上预测 difficulty ranking。

如果：

$$\text{rankcorr}(C(I), 1 - P_{A_{k+1}}(I))$$

仍然高，才说明 measure 不是针对某一个 LLM 拟合出来的。

这比直接把 average LLM accuracy 当 difficulty 强得多。

Cross-class validity

在 LP / facility / scheduling 上拟合，

测试 routing / network / inventory。

否则 “complexity” 可能只是 problem-class classifier。

Representation invariance

对同一个 semantic instance 生成多种 paraphrases:

$$I^{(1)}, I^{(2)}, \dots, I^{(K)}$$

应满足：

$$\mathbf{C}(I^{(1)}) = \dots = \mathbf{C}(I^{(K)})$$

或者至少近似一致。

OPT-Engine 已经提供了一个很好的先例：在 mathematical structure 不变时单独改变 surface language complexity。 ¹⁷

Causal validity

主动改变：

$$\kappa$$

但保持其他 axis 固定。

这是你与 ORAgentBench 最应该拉开的差距。

这项工作怎样形成一条真正有顶会/顶刊潜力的研究线

我不建议第一篇文章试图提出一个类似：

$$C_{\text{operational}} = 0.23N_c + 0.42\kappa + 0.17H + 0.18T$$

的 “万能 scalar” 。

这很容易变成 benchmark-specific regression score，而且 reviewers 会问：

为什么这些权重？换模型还成立吗？换 benchmark 呢？为什么把不同单位的东西线性相加？

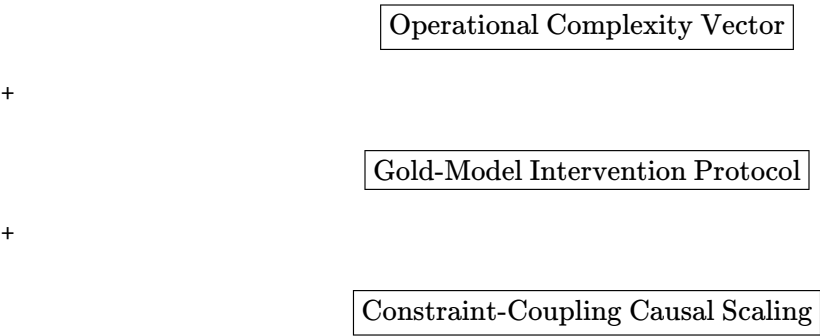
你现在 Q4 选择 vector 是正确的。

第一篇最值得做的是 “定义 + causal benchmark + gold-model decomposition”

我会把中心问题写成：

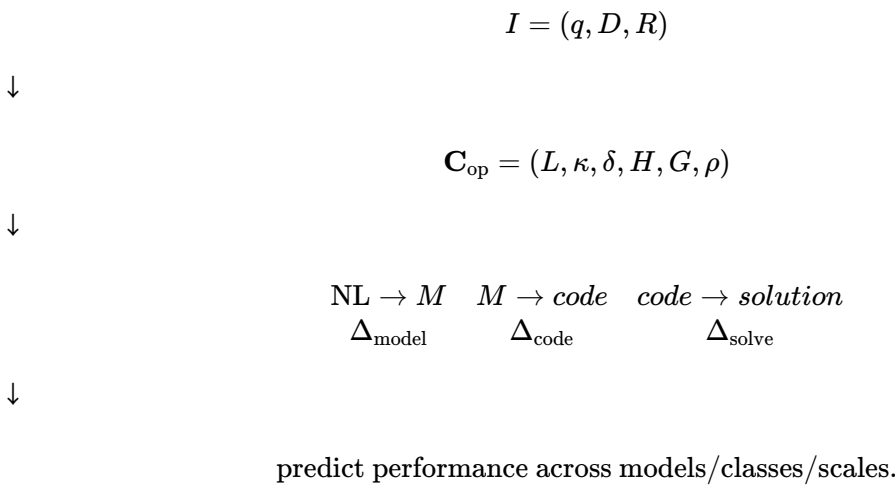
What makes an operational optimization problem intrinsically difficult for solver-integrated LLM agents?

核心贡献可以是：



而不是只提出指标。

论文的 conceptual picture：



这本身已经足够成为一篇 substantial paper。

我认为真正的核心 scientific claim 应该从 Q9 升级成下面这个版本

你现在的 Q9：

对 LLM 来说，正确表达全部 feasibility conditions 比把一个已经正确 formalize 的模型交给 solver 求 optimum 更难。

我建议正式升级为：

Semantic Bottleneck Hypothesis

For solver-integrated LLM systems, within problem regimes admitting standard exact or high-reliability solver realization, end-to-end failure is dominated by semantic formulation error—particularly the omission, distortion, and interaction of feasibility obligations—rather than by model-to-solver compilation or numerical optimization.

然后增加边界命题：

Residual-Solving Boundary Hypothesis

随着 combinatorial scale、nonconvexity、stochasticity、dynamic state 和 algorithmic strategy requirements 增加，gold-model condition 下的 residual solving error 不再接近零，因此 operational complexity 从 modeling-dominated regime 转入 mixed modeling-solving regime。

OPT-Engine 当前的数据支持第一个命题的局部版本，但它自己的 limitation 又暗示第二个命题。 42

这两个命题放在一起，会比直接说 “solver guarantees optimality，因此 computational complexity 不重要” 严谨得多。

第二篇可以做 Complexity-Controlled Benchmark，而不是再做一个普通 benchmark

OPT-Engine 已经证明 controlled scaling 很有吸引力，但它的 constraint manipulation 还比较粗；ORAgentBench 的六维 rubric 比较广，但 dimensions correlated；NLCO taxonomy 很系统，但主要用于 direct combinatorial reasoning。 43

你的下一步可以创建：

OpComplex-Bench

同一个 base operational problem 产生：

$$I_{0,0,0}, I_{1,0,0}, I_{2,0,0}, \dots$$

分别独立 scaling：

$$L, \quad \kappa, \quad \delta, \quad H, \quad G, \quad \rho.$$

这和普通 easy/medium/hard 最大的区别是：

difficulty is generated by interpretable structural interventions rather than assigned retrospectively.

这很适合 ML/benchmark narrative。

第三条线可以做 complexity-aware agents

一旦有：

$$\hat{\mathbf{C}}(I),$$

agent 在求解前先预测：

difficulty profile.

然后根据 profile 选择 strategy:

低 coupling:

single-shot formulation.

高 coupling:

constraint-by-constraint verification.

高 dynamic:

stateful/replanning architecture.

高 ρ :

decomposition/heuristic/warm start.

高 G :

schema extraction/data tooling.

于是 complexity 不再只是 benchmark statistic, 而成为:

agent routing/control variable

ORAgentBench 已经观察到现实 agents 会使用 direct optimization、repair、decomposition、heuristics 和 specialized structure, 而成功轨迹往往需要把 method 与 task structure 匹配。 22

所以这条路线有非常自然的 practical payoff:

$C(I) \rightarrow$ strategy selection.

第四条线可以做 complexity-aware training / curriculum

OptMATH 已经表明可以从 mathematical formulation 出发生成 controllable-complexity data; OR-Instruct 也通过 altering objectives/constraints、rephrasing 和 multiple modeling techniques 增加 training diversity。 44

你的复杂度向量可以进一步用于 curriculum:

$$C_1 < C_2 < \dots < C_K.$$

例如先训练:

$$\kappa \approx 0$$

的 block-independent constraints,

再:

$$\kappa_{\text{chain}},$$

最后：

$$\kappa_{\text{dense}}.$$

然后检验：

$$\text{complexity-aware curriculum} > \text{random sampling}$$

是否提高 OOD modeling。

这会把一个 benchmark paper 变成 learning-method paper。

第五条线可以研究 “LLM complexity 与 human OR expert complexity 是否一致”

这条线其实能把你的工作重新接回 Willemain 1990s 的 OR modeling science。

Willemain 的经典研究强调 model formulation 是 expert craft，而且 expert modelers 在 context、structure、realization、assessment、implementation 之间不断切换。 ¹

未来可以让 OR experts 对同一 instance 记录：

$$T_{\text{human}}, \quad N_{\text{revision}}, \quad N_{\text{constraint omissions}}, \quad \text{confidence}.$$

然后比较：

$$\mathbf{C}(I) \rightarrow \begin{cases} \text{LLM failure} \\ \text{human modeling time/error} \end{cases}$$

如果同一 complexity vector 同时预测 humans 与 LLMs 的 difficulty，那么它才真正接近：

$$\boxed{\text{problem-intrinsic operational modeling complexity}}$$

而不是 “一个碰巧能预测 GPT accuracy 的 benchmark feature”。

我认为这是未来向 OR 顶刊讲故事时非常强的一步。

对论文定位的最终建议

目前这条研究线可以同时形成两种 narrative，但中心不要完全一样。

面向 ICML / NeurIPS / ICLR 一类 ML venue：

重点是：

difficulty measure + controlled benchmark + cross-model prediction + causal intervention + agent adaptation.

也就是：

Current LLM+OR benchmarks report accuracy, but lack an intrinsic, validated theory of instance difficulty.

OPT-Engine、ORAgentBench、NLCO、MIPLIB-NL、IR2Solve 都可以成为非常好的 immediate related work。 45

面向 Operations Research / INFORMS Journal on Computing 一类 OR venue:

重点可以是:

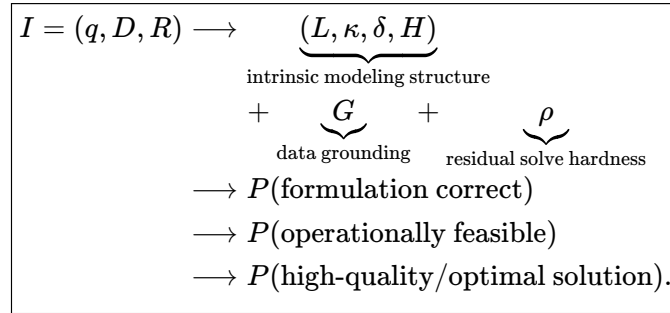
modeling complexity as a formal object

以及:

semantic obligation graph $\rightarrow$ human/LLM modeling reliability $\rightarrow$ solver-integrated operational performance.

这能把经典 model formulation literature、structural CSP complexity、empirical hardness 与 contemporary LLM autoformulation 接起来, 而不是呈现为 “又一个 LLM benchmark”。Willemain 的 modeling science、constraint-structure literature 和 empirical-hardness tradition 恰好提供了三个 foundational anchors。 46

我目前对整个方向的判断可以浓缩成下面这张理论图:



其中最值得优先验证的不是:

“constraint 数越多越难”,

而是:

$$\kappa = \text{constraint interaction/coupling}$$

在控制 constraint count、problem class、language length 和 solver hardness 后, 是否仍然系统预测:

$$P(\text{omitted/mis-modeled feasibility obligations}).$$

与此同时, Gold-Model Intervention 要回答另一个决定性问题:

$$P(\text{success} \mid M^*) \stackrel{?}{\approx} 1$$

以及这个等式在哪些 OR problem classes、instance scales 和 solver regimes 下开始失效。

如果两个实验都得到积极结果，那么 operational complexity 的第一版定义很可能根本不需要从“传统 computational complexity 的翻版”开始，而可以更有针对性地定义成：

$$C_{\text{op}} = (C_{\text{semantic/modeling}}, C_{\text{residual solve}})$$

其中你的主要新贡献是前者，并通过后者确定适用边界。

这比直接寻找一个“constraint number + variable number + runtime”的综合 difficulty score，要更有解释性、更容易因果验证，也更容易同时形成 OR 与 ML 两边都讲得通的研究故事。

1 5 46 Model Formulation: What Experts Think About and When

https://pubsonline.informs.org/doi/10.1287/opre.43.6.916?utm_source=chatgpt.com

2 Exponential Lower Bounds for Polytopes in Combinatorial ...

https://arxiv.org/abs/1111.0837?utm_source=chatgpt.com

3 29 Constraint Satisfaction, Bounded Treewidth, and Finite ...

https://doi.org/10.1007/3-540-46135-3_21?utm_source=chatgpt.com

4 14 15 16 17 18 24 42 43 45 OPT-Engine: Benchmarking the Limits of LLMs in Optimization Modeling via Complexity Scaling

<https://arxiv.org/html/2601.19924v2>

6 9 27 37 Mamo: a Mathematical Modeling Benchmark with Solvers

https://arxiv.org/html/2405.13144v2?utm_source=chatgpt.com

7 38 ORLM: Training Large Language Models for Optimization Modeling

<https://arxiv.org/html/2405.17743v2>

8 IR2Solve: Structured Intermediate Representations for Cost-Efficient Optimization Autoformulation

<https://arxiv.org/html/2608.02641v1>

10 12 25 39 LLM for Large-Scale Optimization Model Auto-Formulation: Bridging Flexibility and Standardization via Agentic Workflow

<https://arxiv.org/html/2601.09635v2>

11 19 20 21 22 23 32 41 ORAgentBench: Can LLM Agents Solve Challenging Operations Research Tasks End to End?

<https://arxiv.org/html/2606.19787>

13 OptiBench Meets ReSocratic: Measure and Improve LLMs ...

https://openreview.net/forum?id=fsDZwS49uY&utm_source=chatgpt.com

26 31 Reasoning in a Combinatorial and Constrained World: Benchmarking LLMs on Natural-Language Combinatorial Optimization

https://arxiv.org/abs/2602.02188?utm_source=chatgpt.com

28 36 Constructing Industrial-Scale Optimization Modeling Benchmark

https://arxiv.org/abs/2602.10450?utm_source=chatgpt.com

30 R-ConstraintBench: Evaluating LLMs on NP-Complete Scheduling

https://arxiv.org/abs/2508.15204?utm_source=chatgpt.com

33 OR-Space: A Full-Lifecycle Workspace Benchmark for Industrial Optimization Agents

https://arxiv.org/abs/2605.28158?utm_source=chatgpt.com

34 SATzilla: Portfolio-based Algorithm Selection for SAT

https://www.jair.org/index.php/jair/article/view/10556?utm_source=chatgpt.com

35 44 OptMATH: A Scalable Bidirectional Data Synthesis Framework for Optimization Modeling

https://arxiv.org/abs/2502.11102?utm_source=chatgpt.com

40 ConstraintBench: Benchmarking LLM Constraint Reasoning on Direct Optimization

https://arxiv.org/abs/2602.22465?utm_source=chatgpt.com